AI Assignment & Deadline Manager

Technical Report | Ollama + Python | Group of Two

Student 1: [NAME / ID] **Student 2:** [NAME / ID]

Submission date: 24 October 2026 **Model:** qwen2.5:3b

Abstract

This project implements a locally running AI Assignment and Deadline Manager for students who must track coursework, estimate effort and prepare study schedules. The application uses Python and Ollama with the qwen2.5:3b language model. Rather than only answering questions, the agent chooses from six callable tools, receives structured results and continues until a request is complete. Assignments are stored in a local JSON file. A greedy earliest-deadline-first planner assigns available study hours to unfinished tasks. The supplied demonstration traces show successful assignment creation, retrieval, workload calculation, study planning, status updates and invalid-date rejection. This report distinguishes observed behavior from limitations and outstanding verification work. It also documents the use of an AI coding assistant, which the group must acknowledge and critically review.

1. Problem and Scenario

Students commonly manage several modules with different submission dates, priority levels and expected workloads. Information may be spread across notes, chats and learning platforms. A simple list can show deadlines but does not automatically calculate a time budget or distribute study hours. Manual planning becomes more difficult when deadlines are close together. The proposed scenario is a student working on a laptop who wants to type natural-language requests such as adding a task, checking what is due soon, or producing a study plan. The project intentionally targets a single user and local execution. This scope is appropriate for a short university assignment because it emphasizes tool use, agent control flow and testing rather than user-interface complexity. The primary success criterion is not a polished website; it is demonstrable execution of real operations with inspectable outputs. A secondary goal is to avoid paid cloud LLM APIs for core inference and to keep assignment records on the local computer.

2. Objectives and Scope

The functional objectives are to create assignments with title, deadline, priority and estimated hours; list tasks by status; identify deadlines in a specified window; update task completion status; calculate workload; and produce a daily study plan. Non-functional objectives include readable terminal interaction, validation of structured inputs, recoverable tool errors, persistent assignment storage, traceable tool execution and local model inference. The system must support a single request that requires more than one operation. For example, a student can ask the agent to check deadlines, calculate workload and plan study sessions in one sentence. Out of scope are calendar synchronization, push notifications, authentication, collaborative editing, automatic extraction of coursework from a university portal, and reliable background scheduling. These exclusions are deliberate: adding them would increase integration effort without strengthening the central agent-loop demonstration.

3. Architecture and Data Flow

The application is separated into an agent controller and deterministic Python tools. The terminal reads a user message and appends it to an in-memory conversation. The controller calls the Ollama chat interface, supplying the conversation and JSON-like function schemas describing available tools and parameters. The model may return a textual answer or one or more function calls. For each function call, the controller looks up the tool name in a fixed allowlist, executes the Python function with the provided arguments and packages the result as

an object containing an ok flag and either data or an error message. This tool result is appended to the conversation and sent to the model on the next iteration. The process continues until the model gives a final response or the controller reaches its ten-round limit. The terminal prints each tool call and tool result, which is essential for a transparent demonstration. Tool activity is also written to agent.log. The architecture avoids treating model-generated prose as evidence that a database operation succeeded: the Python function is the authoritative source of results.

4. Tool Design

The add_assignment tool checks that the title is nonempty and no longer than 120 characters, validates an ISO date, constrains priority to low, medium or high, and accepts estimated hours above zero and at most 100. It assigns an integer identifier, sets the initial status to pending and persists the record. The list_assignments tool filters by all, pending or completed and sorts records by due date. The check_deadlines tool uses a day window and returns pending tasks due before the window ends, including overdue tasks. The update_status tool finds a record by identifier, validates the requested status and saves the changed record. The calculate_workload tool reuses deadline checking and totals the estimated hours. Finally, create_study_plan allocates a daily time budget across pending tasks, sorted first by deadline and then by priority. It returns daily sessions and any hours that could not be allocated. Each tool has a different operation and returns structured data suitable for interpretation by the LLM. The functions are ordinary Python code rather than hidden prompts, making their behavior testable without invoking the model.

5. Agent Loop and Prompt Design

The system instruction tells the model to use tools for task operations, not invent successful outcomes, request missing essential information and explain tool errors. The controller includes the current date with user messages so relative time expressions have a reference point. Function schemas provide descriptions, parameter types, accepted priority and status values, and required fields. The controller appends the assistant tool-call message and then the corresponding tool result message to preserve the context needed for follow-up decisions. The ten-round bound prevents unlimited repeated calls. Conversation history persists while the command-line program remains open, allowing follow-up questions to refer to earlier actions, but it is not saved across program restarts. The model's choice of tool arguments is probabilistic rather than guaranteed. In an initial observed run, a seven-day request triggered check_deadlines with days set to one. A more explicit prompt produced the correct seven-day call. This example illustrates why visible traces and verification of arguments are necessary even when the final natural-language answer appears plausible.

6. Storage, Validation and Error Handling

The persistence layer is a JSON list stored in assignments.json beside the source files. When the file does not exist, loading returns an empty list. When saving, the application serializes the updated list with indentation for readability. JSON was chosen because it is built into Python, easy to inspect during the viva and sufficient for a small single-user demonstration. The tradeoff is that it lacks database transactions and concurrent-update protection. Input checks occur inside tools rather than relying only on the LLM to supply correct values. A malformed date raises a Python validation error, which the controller catches and returns as a structured failure instead of silently creating a bad record. Other handled exceptions include incorrect argument types, missing keys, file operations and JSON decoding problems. Unexpected request-level failures are recorded with a traceback and reported in the terminal. Tool calls and their results are logged to agent.log. Since logs can include student-entered task names, the project should avoid committing runtime logs or private assignment data to a public repository.

7. Study-Planning Algorithm

The planner uses a greedy earliest-deadline-first allocation method. It first sorts pending tasks by due date, breaking ties with high, medium and low priority. It creates a fixed number of daily slots beginning with the

current date. For each day, it starts with the configured daily study-hour budget and allocates as much time as possible to the earliest eligible task. The task's remaining estimated hours and the day's remaining capacity are reduced. Allocation stops when the day's capacity is exhausted or no eligible work remains. A task is not scheduled after its deadline. The output includes a list of dates and sessions, plus a mapping of tasks with hours left unscheduled. In the observed test, a five-hour Database Systems assignment was allocated two hours on October 24, two hours on October 25 and one hour on October 26, before its October 28 deadline. This is a planning suggestion, not a calendar reservation, and it does not account for classes, sleep, other commitments or the student's actual pace.

8. Testing and Observed Results

The group supplied terminal traces from a running agent on October 24, 2026. The add_assignment call saved Database Systems with a deadline of October 28, high priority and five estimated hours, returning ok true and assignment identifier one. The list_assignments call with pending status returned the saved record. A corrected multi-step prompt triggered three calls: check_deadlines with days seven, calculate_workload with days seven, and create_study_plan with days seven and daily_hours two. All three returned ok true. The workload response counted one assignment and five hours, and the planner distributed the hours over three days. A separate invalid-date request used 2026-15-50 and produced ok false with a month-range error, after which the agent asked for a valid date. Another request invoked update_status to mark the assignment completed and list_assignments to retrieve completed work; both succeeded. These are observed manual end-to-end results, not evidence of exhaustive reliability across all prompts. Automated unit tests are included in the starter project and should be rerun by the group before submission, with the exact test command, output and date recorded. The group should additionally inspect agent.log and demonstrate a contextual follow-up to explicitly verify logging and conversation history.

9. Challenges, Limitations and Improvements

The largest challenge is ensuring that a small local model selects the right tool and parameters. Explicit function schemas and a strong system instruction help, but they cannot completely prevent wrong argument choices. The earlier one-day versus seven-day mismatch is a concrete example. Model latency also depends on laptop memory and CPU or GPU capability. The JSON store is suitable for a demonstration but would need a database and transaction handling for multiple users. Conversation history is lost on exit. The planner assumes that estimated hours are accurate and that each day has equal availability; its greedy method is easy to explain but not an optimal personal timetable. A future version could add editable daily availability, overdue warnings, a SQLite database, a graphical interface, persistent conversational state and optional calendar integration. Such integrations should remain separate from the locally running core model and should be added only after more systematic tests.

10. Ethics, AI Assistance and Individual Contributions

An AI coding assistant, ChatGPT, was used to provide an initial Python implementation and to assist with documentation and submission preparation. This assistance must be disclosed transparently. The group is responsible for reading the source, validating its behavior, identifying errors, modifying the implementation where needed, and being able to explain every important design choice during the viva. The project should not be represented as entirely hand-written if it was not. Group membership consists of two students. Student 1 [INSERT NAME AND ID] is responsible for [INSERT ACTUAL CODE, TESTING AND DOCUMENTATION CONTRIBUTIONS]. Student 2 [INSERT NAME AND ID] is responsible for [INSERT ACTUAL CODE, TESTING AND DOCUMENTATION CONTRIBUTIONS]. Before submission, replace these placeholders with truthful details and cite meaningful Git commits made by each person. Both students should demonstrate familiarity with the agent loop, all six tools, the data format, the error-handling strategy and the planning algorithm. If either student has not yet contributed code, the group should allocate and complete genuine changes before claiming compliance.

11. Conclusion

The implemented agent demonstrates a practical use of a locally hosted LLM: it interprets academic planning requests, calls deterministic tools, uses returned data and produces task-focused answers. The provided terminal traces confirm successful creation, retrieval, deadline checking, workload calculation, study planning, status updates and invalid-date handling. The system is intentionally modest, with a command-line interface and JSON persistence, but its clear tool boundary and visible execution trace make it appropriate for explaining agent behavior in a viva. Final submission readiness depends on verifying the repository, unit tests, group contributions, logging, report placeholders and recorded demo rather than on adding more features.